



DEPARTMENT OF COMPUTER SCIENCE

TDT4186 - OPERATING SYSTEMS

Course Information

Spring, 2026

Table of Contents

1	Operating systems	2
1.1	Notes	2
1.2	Learning Objectives	2
2	Processes & Threads	3
2.1	Notes	3
2.2	Learning Objectives	4
3	Scheduling	5
3.1	Notes	5
3.2	Learning Objectives	5
4	Memory	6
4.1	Notes	6
4.2	Learning Objectives	6
5	Page	7
5.1	Notes	7
5.2	Learning Objectives	8
6	Concurrency	9
6.1	Learning Objectives	9
7	Locks	10
7.1	Learning Objectives	10
8	Condition Variables	11
8.1	Notes	11
8.2	Learning Objectives	11
9	Semaphores	12
9.1	Learning Objectives	12
10	I/O Devices	13
10.1	Learning Objectives	13
11	File Systems	14
11.1	Notes	14

11.1.1 Files and File Systems	14
11.1.2 File Naming	14
11.2 Learning Objectives	14
Bibliography	15

Introduction

This booklet presents the syllabus of TDT4186 – Operating Systems (OSs). The course consists of two parts, **theories** (lectures and exercises) and **practical exercises** (hand-on projects).

The textbook we mainly use for this course is:

Remzi H Arpaci-Dusseau and Andrea C Arpaci-Dusseau (2021). *Operating systems: Three easy pieces*. Arpaci-Dusseau Books, LLC

Other textbooks can also be a good complement, e.g.,:

Silberschatz Abraham, Gagne Greg et al. (2014). *Operating System Concepts Essentials*.-2nd

Each section includes the reading chapters and learning objectives for the topics. The reading chapters are the contents we have in our lectures, but other unlisted chapters can also be interesting to read. Learning objectives are organized as a list of questions. There are some contents that are not covered in the textbook, but attending lectures and understanding lectures' slides should be sufficient to understand them.

The code and functions used in the compulsory projects are expected to be understood.

This booklet is intended for 2026.

1 Operating systems

This section is not covered in the textbook.

1.1 Notes

This section mainly introduces the basic concepts of OSs, different OS structures, two modes/spaces, and bootloaders.

Three main structures are discussed in the lecture, but there exist other structure types.

- **Monolithic kernel:** The entire OS is in kernel mode/space¹, including *device driver*, *memory management*, *file systems*, etc. The representative example is Linux.
 - Pros: good isolation, good security guarantee, better performance
 - Cons: inflexibility, reliability, larger size in kernel space
- **Microkernel:** Only the most fundamental services required, e.g., memory management, scheduling, etc, are implemented in kernel mode/space, and other system functions or services, like device drivers, are implemented in user space.
 - Pros: reliability, security, flexibility, smaller size in kernel space
 - Cons: complexity, communication overhead
- **Hybrid kernel:** A hybrid kernel strives to combine monolithic kernel with microkernel. Compared to a microkernel, a hybrid kernel can have some services, functions, or device drivers implemented in the kernel space, which thus differentiates it from a monolithic kernel. As a result, hybrid kernel can be considered as a tradeoff between monolithic kernel and microkernel.
 - Pros and Cons: It inherits the pros and cons from both monolithic kernels and microkernels.

Bootloaders are a new topic introduced in 2026.

1.2 Learning Objectives

Questions you should be able to answer after this chapter

- What are the three OS structures?
- What are the pros and cons of each OS structure?
- Which OS structure do modern OSs use? E.g., Windows, Linux.
- What are the two modes of OSes? Why does an OS need the two modes? How does a process use the two modes?
- What are the design goals for OSs?
- How does a bootloader work?

¹mode and space can be used interchangeably in this context

2 Processes & Threads

Chapter	Name
4	The Abstraction: The Process
5	Interlude: Process API
6	Mechanism: Limited Direct Execution
27	Interlude: Threads

Table 1: Reading chapters

2.1 Notes

Processes and threads are taught together in TDT4186 because they are conceptually similar. All discussions about process management are based on UNIX-like OSs, such as Linux, Ubuntu, and xv6-riscv. Windows, however, uses a different mechanism for managing processes.

Some functions and data structures in xv6-riscv related to compulsory projects can be used for reference.

A **process** in an OS represents a running program and provides a CPU abstraction for executing the program.

A program is a static entity, while its corresponding process is a dynamic entity. OSs utilize three operations to create, execute, and manage processes:

- **fork()**: `fork()` function duplicates a process from the calling process. The calling process is referred to as the *parent*, and the newly created process is called the *child*. Both the parent and child continue execution after the `fork()` function, but the execution order depends on the OS. The child process continues execution from the line where the parent process calls the `fork()` function.
- **exec()**: `exec()` is used to create a new process for a new program. It takes a command as its argument and replaces the content of the current process, such as code and data. By means of `exec()`, a child process for a new program is created.
- **wait()**: `wait()` is a system call that allows a parent process to wait for one of its child processes to terminate. When a child process exits, `wait()` returns the process ID (PID) of that child and optionally provides its exit status. If no child has exited, the calling process is blocked until one does. If the process has no children, `wait()` returns immediately with an error.

Example: Executing a command within a Linux terminal is a representative example of using `fork()`, `exec()`, and `wait()`. Terminal/Shell is a program, so a process is generated when Terminal is launched. The following snippet is from `shell.c` of xv6. Terminal is always waiting for a command, and then some pre-processing will be conducted to determine what kind of command it is receiving. `fork1()` creates a new process, where the return value is used to check whether the executing one is the child process. In the child process, `runcmd()` replaces the content of the parent process with the new command. `wait()` ensures that the child process finishes before the parent process, i.e., Terminal.

```
while (getcmd(buf, sizeof(buf)) >= 0){
    if (buf[0] == 'c' && buf[1] == 'd' && buf[2] == '_'){
        // Clumsy but will have to do for now.
        // Chdir has no effect on the parent if run in the child.
        buf[strlen(buf)-1] = 0; // chop \n
        if (chdir(buf+3) < 0)
            fprintf(stderr, "cannot cd_%s\n", buf+3);
        continue;
    }
    if (fork1() == 0)
        runcmd(parsecmd(buf));
    wait(&r);
}
```

2.2 Learning Objectives

Questions you should be able to answer after this chapter

- What is a process? and what is the main difference between a program and a process?
- What do `fork()`, `exec()`, and `wait()` do?
 - Given a code snippet with these functions, you should know how to analyze the code and decide what is the output.
- What is thread? What are the differences between thread and process?
- What is POSIX? How do some common thread APIs from POSIX work, like `pthread_create()`?

3 Scheduling

Chapter	Name
7	Scheduling: Introduction
8	Scheduling: The Multi-Level Feedback Queue
9	Scheduling: Proportional Share

Table 2: Reading chapters

3.1 Notes

This section introduces different scheduling algorithms. Emphasis is placed on algorithms used in modern OSs, such as RR, MLFQ, and CFS, but the mechanisms, pros, and cons of all algorithms should be understood.

3.2 Learning Objectives

Questions you should be able to answer after this chapter

- What are *turnaround time*, *waiting time*, and *response time*?
- What is starvation in scheduling algorithms?
- What are FIFO, SJF, STCF, RR, MLFQ? What are pros and cons of different scheduling algorithms?
 - Given a job set with all needed information, such as arrival time, execution time, etc, you should be able to compute *average turnaround time* and *average response time/waiting time* for different scheduling algorithms.
- What is the default scheduling algorithm of Linux? How does it work?

4 Memory

Chapter	Name
13	The Abstraction: Address Spaces
15	Mechanism: Address Translation
16	Segmentation
17	Free-Space Management

Table 3: Reading chapters

4.1 Notes

An **address space** is the memory abstraction for a process, providing better isolation between processes and an effective way to manage the memory.

A **memory management unit (MMU)** is a special hardware to facilitate the translations of virtual addresses to physical addresses.

4.2 Learning Objectives

Questions you should be able to answer after this chapter

- What is the address space of a process? What does an address space of a process usually consist of?
- What are virtual address and physical address?
- What are static relocation and dynamic relocation? How do they work respectively?
- What are the two memory allocation methods when allocating contiguous memory space? What are their respective pros and cons? (This is not in the textbook, but you can find this part from the lecture slides.)
- How do different allocation algorithms, such as FF, BF, and WF, work when allocating memory space?
 - Given a set of memory partitions and a set of processes with different memory requirements, you should know what would be the final allocation using a certain algorithm?
- What is segmentation? How does it work?

5 Page

Chapter	Name
18	Paging: Introduction
19	Paging: Faster Translations (TLBs)
20	Paging: Smaller Tables
21	Beyond Physical Memory: Mechanisms
22	Beyond Physical Memory: Policies

Table 4: Reading chapters

5.1 Notes

- **Paging/page mechanism:** the most used virtual memory management mechanism in modern OSs. It divides both virtual memory and physical memory into blocks of the same size.
- **Page:** In paging, blocks of virtual memory are called *pages*.
- **Page frame:** In paging, blocks of physical memory are called *page frames*, or simply *frames*.
- **Page table:** Page table is used to store the mapping of each page to its corresponding page frame. Usually, page table is a per-process structure, i.e., every process has a page table for its address space. For instance, the `proc` data structure has a data structure `pagetable_t`.

In paging, bits of a virtual address are divided into two parts: the bits used to index pages — **page index bits** or **VPN** in the textbook and the bits used for page offset — **page offset bits**.

Page offset bits are determined by the page size, and most modern computer systems are byte-addressable. Then, we can calculate the number of page offset bits as follows:

Page size	Power of two	page offset bits
8 B	2^3 bytes	3 bits
4 KB	2^{12} bytes	12 bits
16 KB	2^{14} bytes	14 bits
1 MB	2^{20} bytes	20 bits

Table 5: The calculation of page offset bits

Page index bits are determined by two factors: the size of the virtual address and the page size.

The size of the virtual address is represented in bits, e.g., 16-bit, 32-bit, or 64-bit. Alternatively, the address space can be expressed as a memory size (e.g., 64 Bytes, 1 MB, 1 GB), which can be converted to bits to obtain the virtual address size.

A straightforward way to determine the number of page index bits is to subtract the number of page offset bits from the size of the virtual address, as shown in Table 6.

Virtual address	Page size	Power of two	page offset bits	page index bits
6-bit	8 B	2^3 bytes	3 bits	3 bits
16-bit	4 KB	2^{12} bytes	12 bits	4 bits
32-bit	16 KB	2^{14} bytes	14 bits	18 bits
64-bit	1 MB	2^{20} bytes	20 bits	44 bits

Table 6: The calculation of page index bits without considering the addressable memory size

However, not all bits allocated to the VPN are required for address translation, because the system does not have to index such a large memory space.

Modern computer systems are typically 64-bit (with some low-end systems still using 32-bit). This raises the question of whether all index bits derived from the straightforward calculation are actually needed.

For example, in a 64-bit system with a 4 KB page size, there would be 52 bits for the page index, theoretically allowing the system to address 2^{64} bytes of memory—i.e., **16 exabytes**. In practice, such an enormous address space is unnecessary. Therefore, when determining the number of page index bits, system designers consider the realistic memory requirements of processes.

In xv6-riscv, the **Sv39 RISC-V** format is used, where only 39 out of the 64 bits are utilized for virtual addressing, and the remaining 25 bits are unused. These 39 bits are sufficient to address up to 512 GB of memory, which already far exceeds the physical memory available in most current systems.

In this case, 512 GB represents the addressable memory size chosen by the system designers, as it is sufficient for most practical workloads in the foreseeable future.

The least significant bits are typically used for address translation.

Most OSs deploy per-process page tables, where each process maintains its own page table. This approach provides better memory isolation between processes. In contrast, an inverted page table uses a system-wide page table that stores translations for all page frames (physical memory).

5.2 Learning Objectives

Questions you should be able to answer after this chapter

- What is paging in an OS? How does paging work?
- How does an OS use a virtual address for page translation?
 - Given a virtual address space and page size, how many bits are needed for the page number and offsets, respectively?
 - Given a memory system with complete paging information, you should be able to translate given virtual addresses into corresponding physical addresses.
- What is a page table?
- What are the two main problems of a paging system without hardware assistance, and how can they be addressed?
- What is a TLB? Why is a TLB needed in a paging system?
- What is page replacement? What are page hits and page misses?
- How do different page replacement policies/algorithms work, e.g., OPTIMAL, FIFO, LRU, etc?
 - Given a page replacement policies and a page access sequence, you should be able to calculate the average memory access time.

6 Concurrency

Chapter	Name
26	Concurrency: An Introduction
27	Interlude: Thread API

Table 7: Reading chapters

6.1 Learning Objectives

Questions you should be able to answer after this chapter

- What is the concurrency issue?
 - Given a code snippet with threads, you should be able to analyze the output and identify the possible issues.
- What is the critical-section problem? What requirements should a solution for the critical-section problem satisfy?

7 Locks

Chapter	Name
28	Locks

Table 8: Reading chapters

7.1 Learning Objectives

Questions you should be able to answer after this chapter

- What is a lock in an OS?
- Why do locks need hardware instructions, like `Test-And-Set()` and `Compare-And-Swap()`, for their implementation?
- What is a spinlock? What are the advantages and disadvantages of spinlocks? What methods can be used to address their performance issues?
- What are ticket locks and queue locks?

8 Condition Variables

Chapter	Name
30	Condition Variables

Table 9: Reading chapters

8.1 Notes

Conditional variables are often used in conjunction with locks in order to prevent race conditions.

8.2 Learning Objectives

Questions you should be able to answer after this chapter

- What is a condition variable? Why do we need condition variables?
- What is the producer/consumer problem? How can it be solved using condition variables?
 - Given a code snippet, you should be able to analyze it.

9 Semaphores

Chapter	Name
31	Semaphores
32	Common Concurrency Problems

Table 10: Reading chapters

9.1 Learning Objectives

Questions you should be able to answer after this chapter

- What is a semaphore? What operations do semaphores support?
- How can semaphores be used to implement solutions for mutual exclusion, ordering, and the P/C problem?
 - Given a code snippet, you should be able to analyze it.
- What is a deadlock? What are the four necessary conditions for deadlocks?

10 I/O Devices

Chapter	Name
36	I/O Devices

Table 11: Reading chapters

10.1 Learning Objectives

Questions you should be able to answer after this chapter

- Why do I/O systems deploy a hierarchical structure?
- What are ports, buses, controllers, and device drivers?
- What are I/O interrupt and polling? Pros and cons.
- What are I/O instructions and memory-mapped I/O?
- What are programmed I/O and DMA?

11 File Systems

Chapter	Name
39	Interlude: Files and Directories
40	File System Implementation

Table 12: Reading chapters

11.1 Notes

11.1.1 Files and File Systems

File system: A method and structure for managing storage devices, such as HDDs, SSDs, and external storage devices (USB flash drives). It facilitates the organization, access, retrieval of files and directories.

11.1.2 File Naming

Three types of naming for files:

- **inode:** A data structure that stores the metadata of a file. Each inode has a unique number, similar to a process ID, and every file on disk is associated with an inode. Files can be accessed using their inode numbers.
 - Cons: A modern computing system may have thousands or millions of files, making it impractical for users to remember inode numbers and use them directly to manipulate files.
- **path:** A human-readable identifier used to access files, e.g., `/user/test.txt` in Unix-like systems or `C:\Windows\test.txt` in Windows.
 - A file's path is associated with an inode, allowing the OS to locate the file using the path name.
 - The mapping between a path and its inode is stored in the directory that contains the file.
- **file descriptor:** A run-time identifier (an unsigned integer) used by a process to reference an open file.
 - Each process maintains a file descriptor table to keep track of all open files.
 - By default, each process has three file descriptors: 0 for STDIN, 1 for STDOUT, and 2 for STDERR.

11.2 Learning Objectives

Questions you should be able to answer after this chapter

- What are files and file systems?
- What are the three identifiers an OS uses for a file?
- What do common file operations (e.g., `open()`, `read()`, etc.) do? What is the *offset* of a file, and how do different file operations affect it?
 - Given a code snippet, you should be able to analyze it.
- How is the on-disk structure organized?
- What is a directory?
- How do different file operations access the on-disk structure?

Bibliography

Abraham, Silberschatz, Gagne Greg et al. (2014). *Operating System Concepts Essentials*.-2nd.
Arpaci-Dusseau, Remzi H and Andrea C Arpaci-Dusseau (2021). *Operating systems: Three easy pieces*.
Arpaci-Dusseau Books, LLC.